Movie recommender system

Adam Sambor, Jakub Żuk

April 2025

1 Introduction

The aim of this project is to build the movie recommender system. We try to predict user's rating of certain movies based on training set from IMDB database. The movies are rated from 0.0 to 5.0 with step of 0.5 using four methods: NMF, SVD, SVD2 (iterated SVD) and SGD which will be discussed further.

Our database contains about 600 users and about 9000 movies, but only about $100000 \ll 600 \cdot 9000$ ratings, so one of the main challenges of this project is to fill somehow the large chunk of missing data, because three out of four algorithms need full rating matrix.

Work with NMF, SVD and iterated SVD algorithms was done with the heavy use of scikit-learn python library while SGD implementation used torch library.

2 Training of the model

Model training was performed on the given *ratings.csv* file. Important part of training the model will be finding best choice of dimension r of matrices used to compute $Z_{\text{approx}} = WH$ where W is $n \times r$ and H is $r \times d$ matrix.

To find the optimal parameters r (and λ for SGD), we used k-fold cross-validation. For each method, we split our training set into 10 folds and every of them to train set (about 80% of the fold) and test set (about 20%).

The search of these parameters was optimized for Root Mean Square Error (RMSE). In each fold we trained models on train set and compute RMSE using test set. We plotted obtained RMSEs on box plots and analyzed them quantitatively and qualitatively to find best parameter (one with the smallest RMSE).

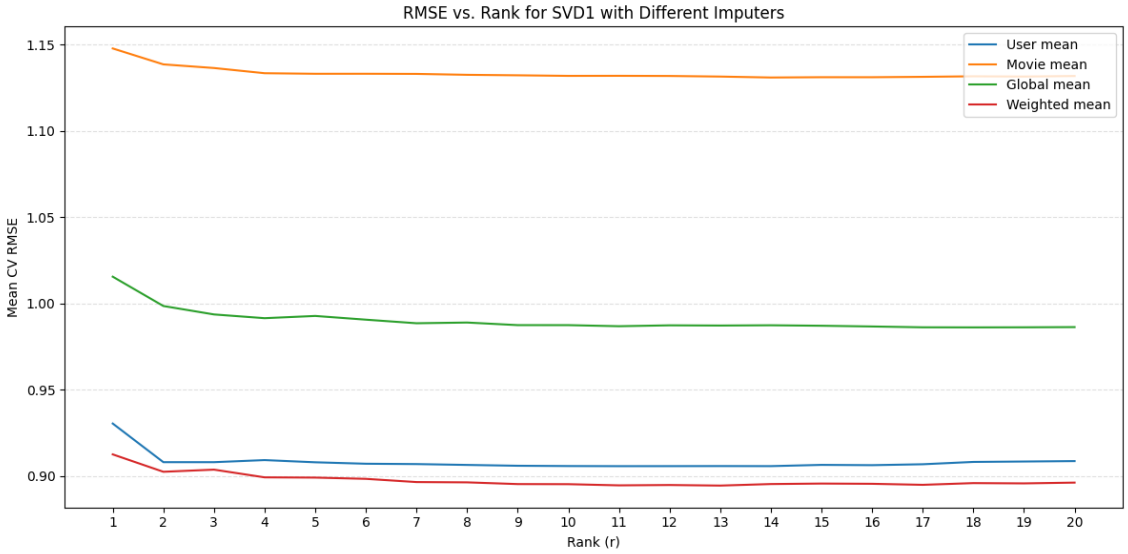
3 Data imputation

To perform three out of four presented algorithms we'd like to have complete data, so we'd like to have ratings of every movie by every user. There are many approaches how to do this, but knowing only the ratings of said user and said movie we found the following approach appropriate:

Let us consider user i and movie j. We want to model the following phenomena:

- ratings of users who rated many movies typically better reflect expected rating
- estimated rating should be combination of user's previous ratings and movie's previous ratings
- if the movie is very good or very bad, there are not very many mediocre ratings

We came up with the idea to reflect these features using slightly modified weighted mean of user's previous ratings median and movie's previous ratings median. This way we catch dependency on both users and movies, we don't fall into the trap of giving mediocre ratings to controversial movies ("You either love it or hate it" movies) and we pay more attention to user's previous ratings than overall movie rating thanks to use of weighted mean.



Comparison of different methods of data imputation

As we can see, our method of imputation works really well in comparison to more standard approaches. The interesting observation is how good imputing using user mean is. It turns out that the criticism of the user plays a much bigger role than other users' ratings of the film.

Of course there are better ways to model this situation, including more sophisticated ones (for example finding similarities between movie ratings or user ratings), but using them brings only slight improvement and costs much more computation time, however redoing this project having more time, computation power and maybe richer database (for example including movie genre) and using these methods could be an interesting task.

4 Prediction methods

4.1 NMF

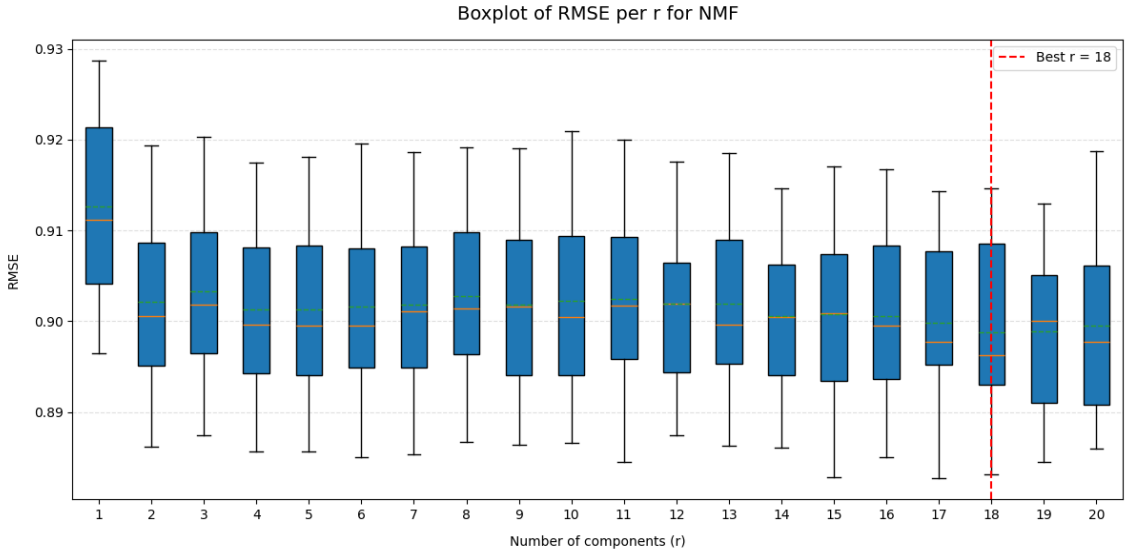
Non-negative Matrix Factorization

NMF approximates the matrix Z of size n × d as

$$\mathbf{Z} \approx \mathbf{WH}$$

where all three (Z, W, H) matrices have all non-negative entries. W is of size n × r and H is of size r × d, with r as a parameter.

To perform NMF algorithm, we need to have complete data, so we need to impute missing entries of matrix Z. We do this using method presented in previous section. NMF is quite simple, so it probably doesn't need detailed description. Because of restrictions on W and H matrices, we expect NMF to perform the worst of all algorithms.



Box plot showing RMSE for certain values of r

4.2 SVD1

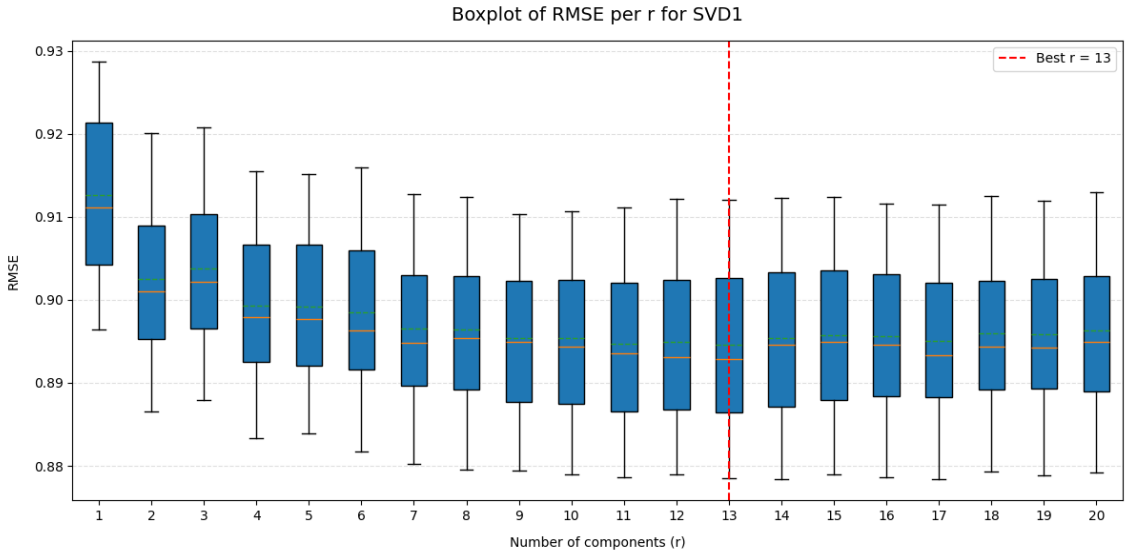
Singular Value Decomposition

The approach here is very similar to NMF. SVD decomposes matrix Z to product of three matrices:

$$\mathbf{Z} \approx Z_r = \mathbf{U}_r \Lambda_r \mathbf{V}_r^T$$

where the three matrices are obtained from truncated SVD (truncation of SVD up to dimension r which is our parameter).

We can observe, that denoting $\mathbf{W} = \mathbf{U}_r$ and $\mathbf{H} = \Lambda_r \mathbf{V}_r^T$ we obtain form analogous to NMF. Abandoning the condition of non-negativity of the matrix leads to the expected greater accuracy of approximation using SVD1 than NMF. Again we need to impute missing values of Z matrix to be able to perform this algorithm.



Box plot showing RMSE for certain values of r

4.3 SVD2

This approach applies SVD recursively to improve the approximation.

Iterated Singular Value Decomposition

Starting with the initial decomposition $\mathbf{Z} \approx \mathbf{U}_r^{(1)} \Lambda_r^{(1)} (\mathbf{V}_r^{(1)})^T$, we compute the residual matrix $\mathbf{R} = \mathbf{Z} - \mathbf{U}_r^{(1)} \Lambda_r^{(1)} (\mathbf{V}_r^{(1)})^T$. Subsequent SVDs are then performed on the residual:

$$\mathbf{Z} \approx \sum_{k=1}^{K} \mathbf{U}_r^{(k)} \Lambda_r^{(k)} (\mathbf{V}_r^{(k)})^T$$

where K iterations progressively refine the approximation.

SVD2 (iterated SVD) is the upgraded version of normal SVD. It's idea is to compute truncated SVD multiple times to achieve better approximation. Of course we need to impute missing values of Z matrix when we use iterated SVD approach.

We expect RMSE of SVD2 to be the smallest of all three discussed methods.

Problem setup

Let $\Omega \subset \{1, \dots, n\} \times \{1, \dots, d\}$ be the set of observed entries. We seek

$$\min_{X \in \mathbb{R}^{n \times d}} \sum_{(i,j) \in \Omega} \big(Z_{ij} - X_{ij}\big)^2.$$

Iterated-SVD idea

1. Impute missing entries of matrix Z (like in section 3).
2. For $t = 0, 1, 2, \dots$ until convergence:
   (a) Compute the rank-r truncated SVD of the filled matrix $X^{(t)}$:

$$X^{(t)} = U^{(t)}\,\Lambda^{(t)}\,(V^{(t)})^T \quad \longrightarrow \quad X_r^{(t)} = U^{(t)}_{[:,1:r]}\,\Lambda^{(t)}_{1:r,1:r}\,(V^{(t)}_{[:,1:r]})^T.$$

   (b) Define $X^{(t+1)}$ by keeping observed entries from Z and filling missing ones from the low-rank reconstruction:

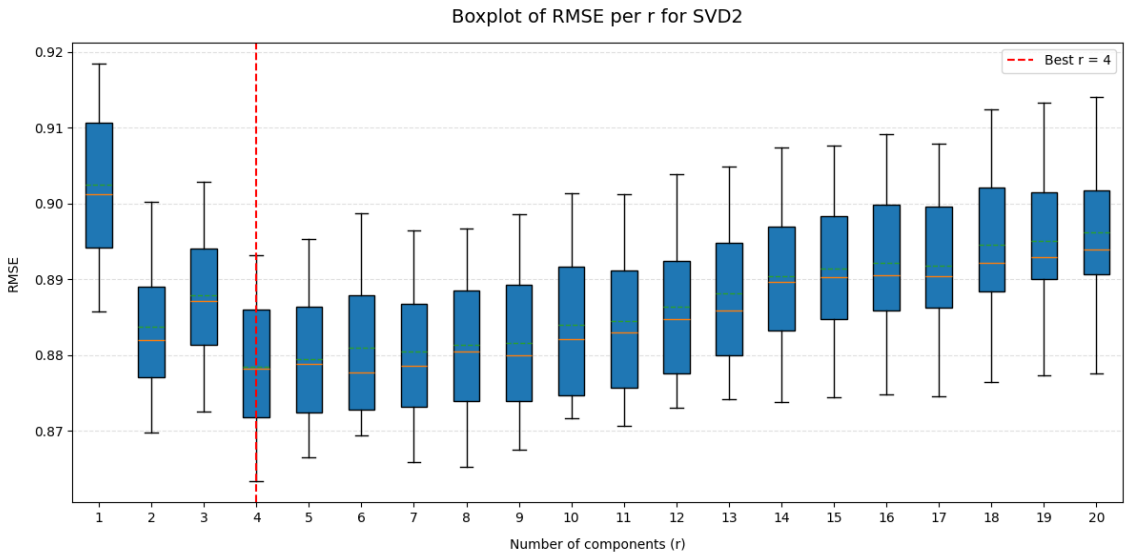
$$X_{ij}^{(t+1)} = \begin{cases} Z_{ij}, & (i,j) \in \Omega, \\ (X_r^{(t)})_{ij}, & (i,j) \notin \Omega. \end{cases}$$

Below we put it in algorithmic way:

Algorithm 1 Iterated SVD (SVD2)

Require: Observed matrix $Z \in \mathbb{R}^{n \times d}$, observed set Ω, rank r, iterations T, imputation function impute.

Initialize $X^{(0)}$ by filling missing via impute.
for $t = 0, 1, \dots, T-1$ **do**
  Compute truncated SVD: $X^{(t)} = \widetilde{U}^{(t)}\,\Sigma^{(t)}\,(\widetilde{V}^{(t)})^T$.
  Set $W^{(t)} = \widetilde{U}^{(t)}$, $H^{(t)} = \Sigma^{(t)}(\widetilde{V}^{(t)})^T$.
  Reconstruct $\widehat{X}^{(t)} = W^{(t)}H^{(t)}$.
  Re-inject observed: for all (i,j), $X_{ij}^{(t+1)} = Z_{ij}$ if $(i,j) \in \Omega$, else $\widehat{X}_{ij}^{(t)}$.
end for
return Final approximation $X^{(T)}$.



Box plot showing RMSE for certain values of r

4.4 SGD

While all previous algorithms were somehow similar, stochastic gradient descent comes with completely different approach to our problem. First of all, we do not need to impute missing data, because we do not perform typical matrix decomposition. Our goal will be to start with any random matrices W and H and modify them in many iterations to minimize initially defined loss function.

Stochastic Gradient Descent

For a given Z of size $n \times d$, we seek matrices W (size $n \times r$) and H (size $r \times d$) by solving:

$$\text{argmin}_{W,H} \frac{1}{N} \sum_{(i,j):z_{ij}\neq ?} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2 + \lambda(||\mathbf{w}_i||^2 + ||\mathbf{h}_j||^2)$$

where $\mathbf{w}_i^T$ is the i-th row of matrix $\mathbf{W}$ and $\mathbf{h}_j$ is the j-th column of matrix $\mathbf{H}$.

We performed SGD using PyTorch and the built-in SGD optimizer to minimize the loss function. In theory, SGD seem very promising because it does not need data imputation, but in practice, it turns out that this algorithm is very sensitive to small parameters fluctuations and performing well on certain dataset could result in fatal predictions on other set of data. Below we shall discuss SGD in detail.

General idea behind Gradient Descent

We wish to minimize an empirical risk (loss) over a training set $\{(x_i, y_i)\}_{i=1}^N$:

$$\min_{\theta \in \mathbb{R}^d} L(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(x_i;\theta), y_i\big) \;=\; \frac{1}{N} \sum_{i=1}^{N} L_i(\theta)$$

where:

- θ are the model parameters,
- $\ell(\cdot, \cdot)$ is a per-sample loss,
- $L_i(\theta) = \ell(f(x_i;\theta), y_i)$.

Standard Gradient Descent Recap

The (batch) gradient descent update is:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) = \theta_t - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\theta_t) \tag{1}$$

where $\eta > 0$ is the learning rate.

Stochastic Gradient Descent (SGD)

Instead of computing the full gradient over all N samples like in standard gradient descent, SGD uses a single (or small mini-batch) sample at each iteration to form an unbiased estimate of the full gradient.

• Unbiased Estimator of the full gradient

Pick (i_t, j_t) uniformly from $\{1, \ldots, n\} \times \{1, \ldots, d\}$. Define

$$g_t^W = \nabla_{W_{i_t,:}} L_{i_t j_t}(W, H) = \frac{2}{N} \left(\langle W_{i_t,:}, H_{:,j_t} \rangle - Z_{i_t j_t}\right) H_{:,j_t},$$

$$g_t^H = \nabla_{H_{:,j_t}} L_{i_t j_t}(W, H) = \frac{2}{N} \left(\langle W_{i_t,:}, H_{:,j_t} \rangle - Z_{i_t j_t}\right) W_{i_t,:}^T.$$

Then

$$\mathbb{E}[g_t^W \mid W, H] = \nabla_W L, \quad \mathbb{E}[g_t^H \mid W, H] = \nabla_H L.$$

• Update of the parameter

The parameter update becomes:

$$\theta_{t+1} = \theta_t - \eta_t \, g_t = \theta_t - \eta_t \, \nabla L_{i_t}(\theta_t), \tag{2}$$

where η_t may be constant or follow a decay schedule.

Algorithm 2 SGD for low-rank Matrix Approximation

Require: Target matrix $Z \in \mathbb{R}^{n \times d}$, rank r, initial factors $W^{(0)} \in \mathbb{R}^{n \times r}$, $H^{(0)} \in \mathbb{R}^{r \times d}$, learning rates $\{\eta_t\}_{t=0}^{T-1}$, iterations T.

1: **for** $t = 0, 1, \ldots, n\_epochs$ **do**

2: Sample (i_t, j_t) uniformly from $\{1, \ldots, n\} \times \{1, \ldots, d\}$

3: Compute error: $LOSS_t = \frac{1}{N} \sum_{(i,j):z_{ij} \neq ?} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2 + \lambda(||\mathbf{w}_i||^2 + ||\mathbf{h}_j||^2)$

4: Gradient $W_{i_t,:}$: $g_t^W = \frac{2}{N} LOSS_t \, H_{:,j_t}^{(t)}$

5: Gradient $H_{:,j_t}$: $g_t^H = \frac{2}{N} LOSS_t \, (W_{i_t,:}^{(t)})^T$

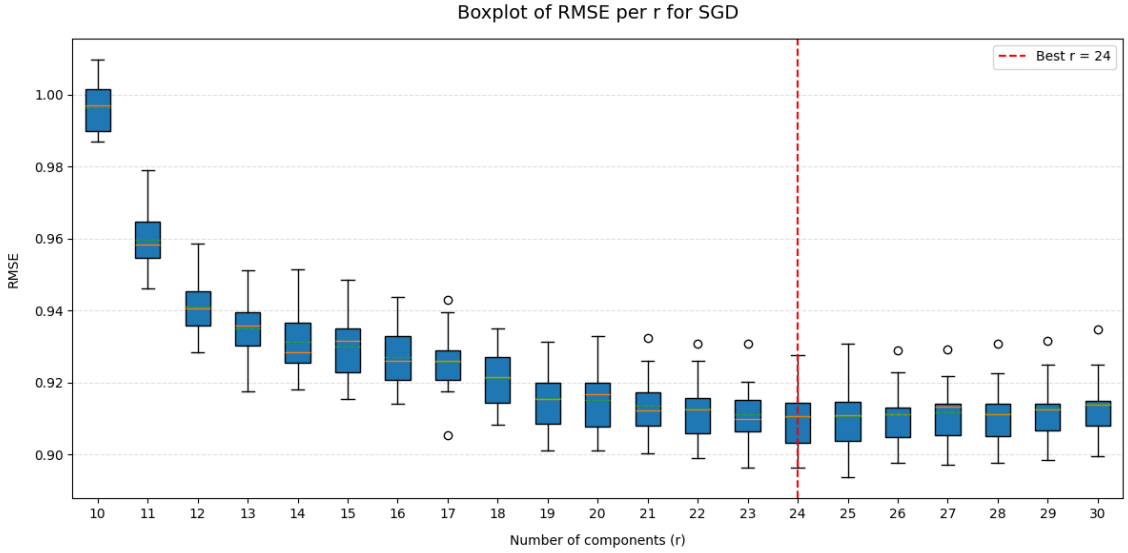
6: Update:

$$W_{i_t,:}^{(t+1)} = W_{i_t,:}^{(t)} - \eta_t \, g_t^W,$$
$$H_{:,j_t}^{(t+1)} = H_{:,j_t}^{(t)} - \eta_t \, g_t^H$$

7: Keep all other rows/columns unchanged.

8: **end for**

9: **return** $W^{(T)}, H^{(T)}$



Box plot showing RMSE for certain values of r with $\lambda = 0$

After testing with the use of grid search for the optimal hyperparameter λ, it turns out that the best is $\lambda = 0$. Values of r smaller than 10 were only worse.

5 Results

Method	Optimal parameters values	RMSE
NMF	$r = 18$	0.8987
SVD1	$r = 13$	0.8946
SVD2	$r = 4$	0.8785
SGD	$r = 24, \lambda = 0$	0.9104

Despite the best theoretical background, SGD performed the worst of all algorithms. As we stated earlier, it is extremely hard to choose the right parameters (r, λ, learning rate, starting point, number of epochs), so that is why in our case SGD performed the worst. The iterated SVD was better than the standard truncated SVD which was slightly better than NMF, so it is in line with expectations.